



WELCOME





Analyzing Algorithm Efficiency : Space complexity

Analyzing Algorithm Efficiency

Agenda

Space Complexity

Key Takeaways



Space Complexity

Learning about Space Complexity

Introduction

Space Complexity

Measures the **memory** an algorithm
uses relative to the **input** size.

Example - Constant Space

Code

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;
    cout << "Sum: " << sum << endl;
    return 0;
}
```

Constant space

$O(1)$

Example - Linear Space

Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        arr[i] = i;
    }
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

The program reads an integer **n** from the user and creates a vector of **size n**, so the **space** used grows **linearly** with **n**.

O(n)

Example - Linear Space

Code

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<vector<int>> matrix(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            matrix[i][j] = i * n + j;
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

The program creates a 2D vector (**matrix**) of size **n*n**. The space used grows **quadratically** with **n**.

$$O(n*n)$$

Example - Function

Code

```
int addition(int a, int b){  
  
    int c= a + b;  
  
    return c;  
}
```

What are the variables the
function is storing?

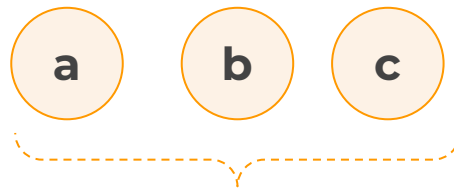


Function

Code

```
int addition(int a, int b){  
  
    int c = a + b;  
  
    return c;  
}
```

These are the **three** variables
that need to be **stored**.



Number of memory spaces = 3

Function

A was assigned 4, but now it has 20, 30, 40, and 1 lakh instead.

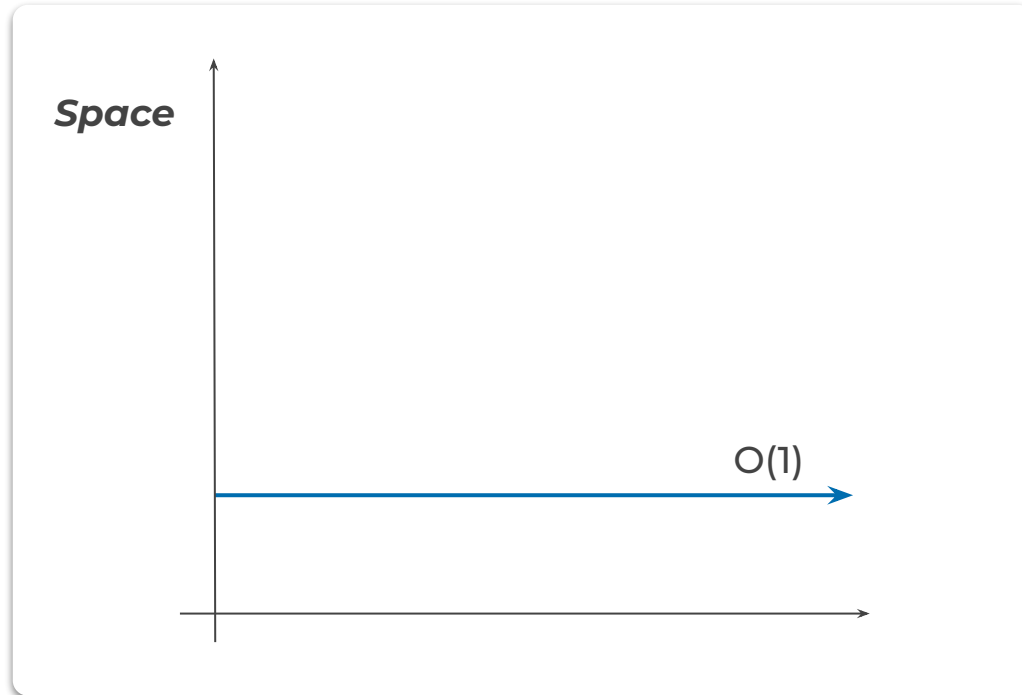
		Number of memory spaces (a, b, c)
a = 4	b = 5	3
a = 20	b = 5	3
a = 30	b = 5	3
a = 40	b = 5	3
1 Lakh	b = 5	3

Constant

[$O(3) \approx O(1)$]

Space Complexity

Function



Function

Code

```
int addition(int a, int b){  
  
    int c= a + b;  
  
    return c;  
}
```



Is there anything we can do
to **reduce space**?

Function

Code

```
int addition(int a, int b){  
    return(a + b);  
}
```

a and **b** are required. Instead of creating variable **c**, directly **return (a + b).**

Array-Function

Code

```
int addarray(int arr[], int n){  
  
    int sum=0;  
  
    for(int i=0; i<n; i++){  
  
        sum = sum + arr[i];  
  
    }  
  
}
```

Give the **sum of all the elements**
in the array.

Input

arr = 5, 7, 10

Array-Function

Code

```
int addarray(int arr[], int n){  
    int sum=0;  
    for(int i=0; i<n; i++){  
        sum = sum + arr[i];  
    }  
    return sum;  
}
```

Input space

$\left\{ \begin{array}{l} arr[] \\ n \end{array} \right\} O(n)$

Auxiliary space

$\left\{ \begin{array}{l} sum \\ i \end{array} \right\} O(1)$

Array-Function

Code

```
int addarray(int arr[], int n){  
  
    int sum=0;  
  
    for(int i=0; i<n; i++){  
  
        sum = sum + arr[i];  
    }  
    return sum;  
}
```

Total space = **Input space** + **Auxiliary space**

= **$O(n)$** + **$O(1)$**

= **$O(n)$**

Total space = $O(n)$

Key Takeaways

- ✓ **Space Complexity** – Refers to how much memory an algorithm uses, including input space and extra (auxiliary) space.
- ✓ **Efficiency Focus** – We compare algorithms based on how they scale, not exact values, to choose the most optimal one for large inputs.





THANK YOU





ALL THE BEST

